

Operating systems

Lecture #4

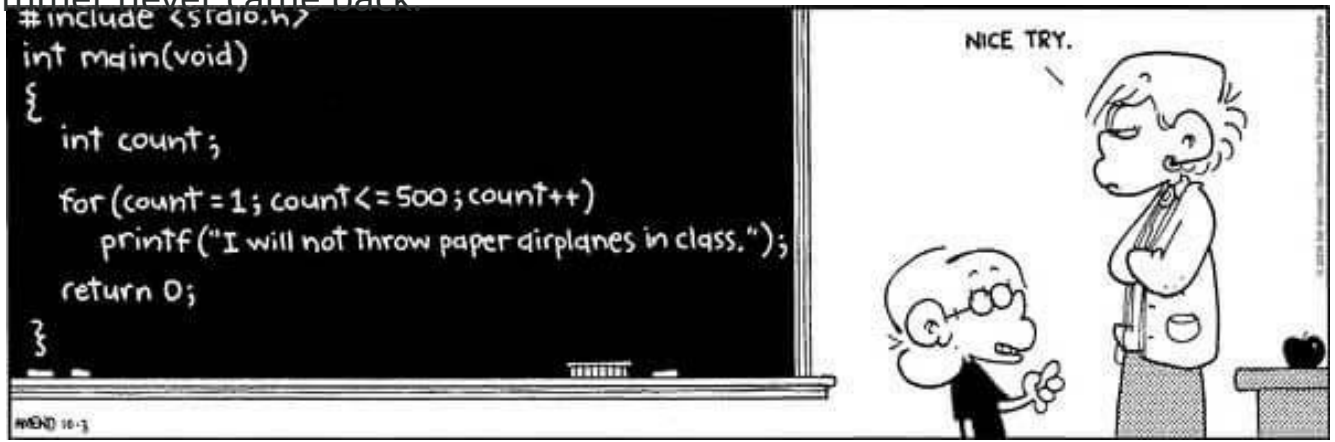
Content Lecture#4

01	File Descriptor
02	Standard File Descriptor
03	InterProcess COmmunication
04	Pipes
05	Descriptor Table

Lets take a Break

The wife asks: "Run to the store and pick up a loaf of bread. While you are there, get eggs."
The programmer never came back.





A programmer's wife tells him: "Run to the store and pick up a loaf of bread. And if they have eggs, get a dozen"
The programmer comes home with 12 loaves of bread

Revising Previous Lecture

Important System Calls Used in OS

fork()
Processes use this system call to create processes that are a copy of themselves. With the help of this system Call parent process creates a child process, and the execution of the parent process will be suspended till the child process executes.

exec()
This system call runs when an executable file in the context of an already running process that replaces the older executable file. However, the original process identifier remains as a new process is not built, but stack, data, head, data, etc. are replaced by the new process.

exit()
The exit() system call is used to terminate program execution. Specially in the multi-threaded environment, this call defines that the thread execution is complete. The OS reclaims resources that were used by the process after the use of exit() system call.

kill():
The kill() system call is used by OS to send a termination signal to a process that urges the process to exit. However, a kill system call does not necessarily mean killing the process and can have various meanings.

wait()
In some systems, a process needs to wait for another process to complete its execution. This type of situation occurs when a parent process creates a child process, and the execution of the parent process remains suspended until its child process executes.
The suspension of the parent process automatically occurs with a wait() system call. When the child process ends execution, the control moves back to the parent process.



Inter Process Communication

Cooperating process can affect or be affected by other processes, including sharing data reasons for cooperating processes:

File Descriptor

A file descriptor is **an unsigned integer used by a process to identify an open file**.

Standard Descriptor

- When a file or a device opens an integer is given as descriptor. And through this descriptor file is accessed. 3 descriptors are fixed

File descriptor #	Purpose	Posix Name	Stdio stream
0	Standard input	STDIN_FILENO	stdin
1	standard output displayed using printf();	STDIN_FILENO	stdout
2	standard error displayed using perror();	STDIN_FILENO	stderr

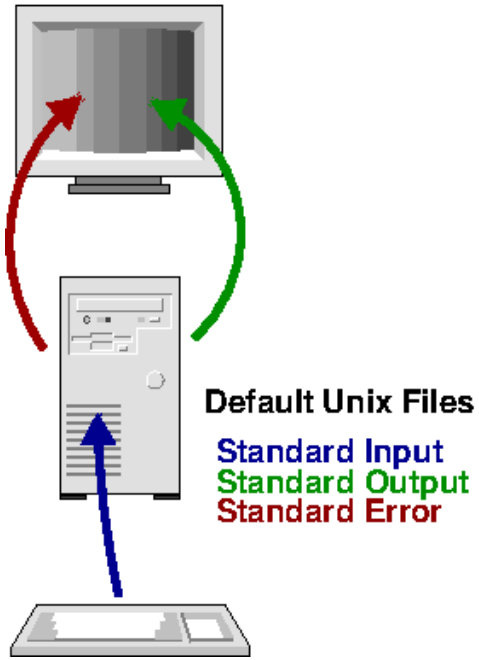
- A descriptor table is associated with every process and PCB holds its pointer. Standard descriptor can also be replaced for any other file and done by write() system file.

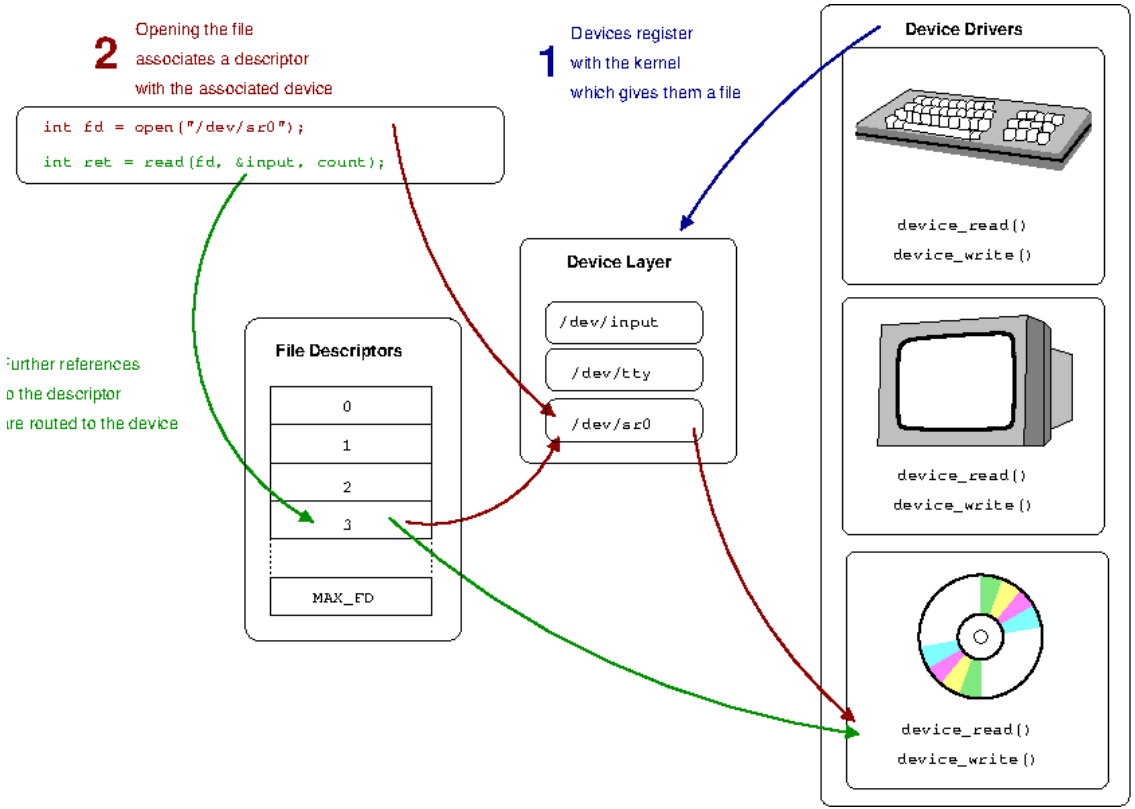
- Size_t Write(int fd, void* buf, size_t cnt)

- Parameter
  - Fd: file descriptor
  - Buf: buffer to write data to(add of the buffer)
  - Cnt: length of the buffer

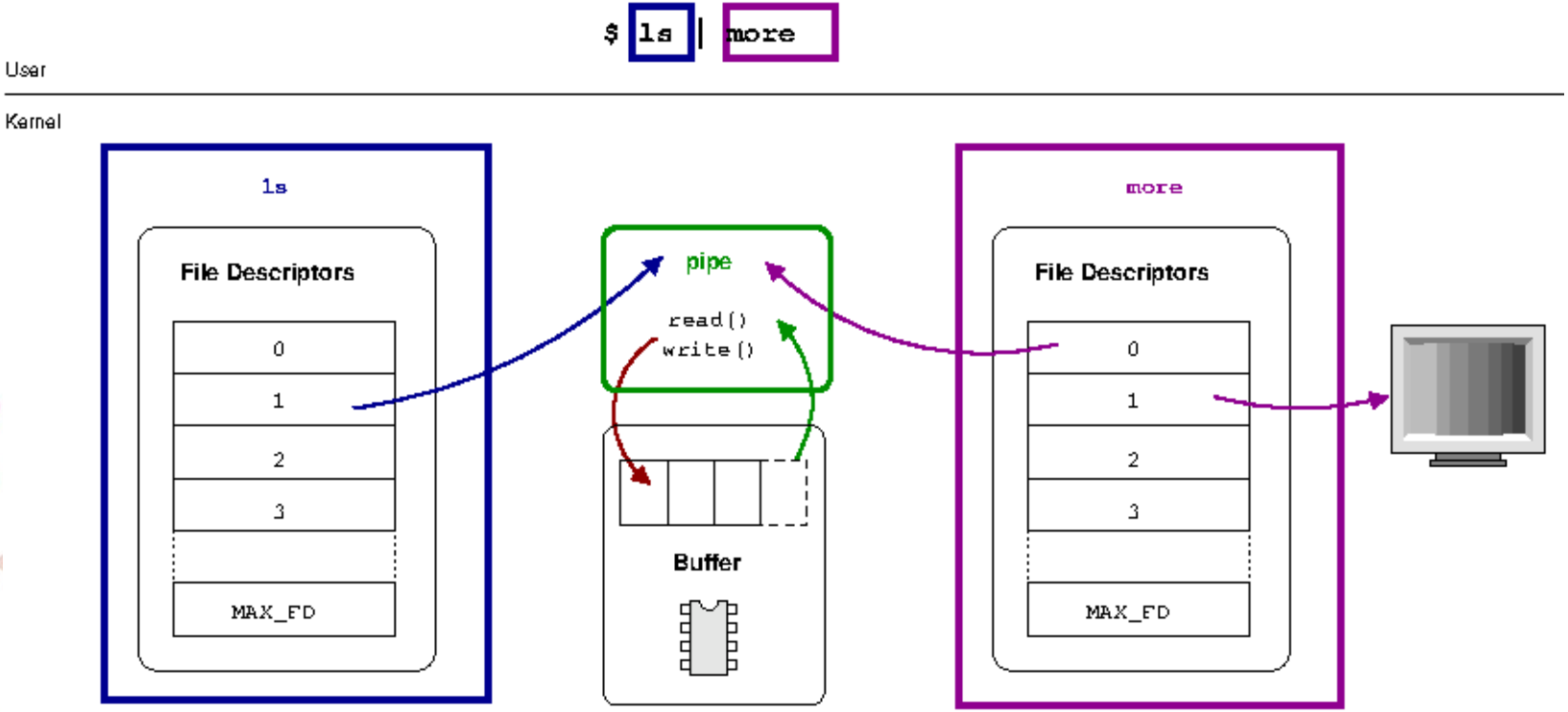
- Returns How many bytes were actually written
  - Return Number of bytes written on success
  - Return 0 on reaching end of file
  - Return -1 on error
  - Return -1 on the signal interrupt

File descriptor





File Descriptor



Standard Descriptor

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/wait.h>

int main(int argc, char const *argv[])
{
        char array[100];
        int n = read(0,array,100);

        write(1,array,n);

        return 0;
}
```

https://www.onlinegdb.com/online_c_compiler#

100 is passed

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/wait.h>

int main(int argc, char const *argv[])
{
        char array[100];
        int n = read(0,array,100);

        write(1,array,100);

        return 0;
}
```

2 is passed for error so programmer know if this is output or error

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/wait.h>

int main(int argc, char const *argv[])
{
        char array[100];
        int n = read(0,array,100);

        write(2,array,100);

        return 0;
}
```

Read 5 bytes but diplay 100 bytes

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/wait.h>

int main(int argc, char const *argv[])
{
        char array[100];
        int n = read(0,array,5);

        write(1,array,100);

        return 0;
}
```

Inter Process Communication

Cooperating process can affect or be affected by other processes, including sharing data reasons for cooperating processes:

- Information sharing
- Computation speedup
- Modularity
- Convenience

Cooperating processes need inter-process communication (IPC)

- Methods to communicate among processes and called inter Process communication

  - **Pipes** (Anonymous pipes (used for IPC between parent child or siblings)
  - **Named Pipes (FIFO)**file created on disk every king of Process can communicate through this
  - **Shared memory** (memory created among communicating process and shared area is created by any one of the process used for bulk data)
  - **Socket** (Two process runs on different machines)
  - **Message Passing** Message passing is a high level IPC mechanism Message is a piece of information that is passed from one process to another

Inter Process Communication

Pipes

- A mechanism in which the output of one process is directed into input of another
- Pipe is one-way communication only i.e we can use a pipe such that One process write to the pipe, and the other process reads from the pipe. It opens a pipe, which is an area of main memory that is treated as a “virtual file”.
- A pipe file is created by using pipe() system call
- A pipe can be accessed like an ordinary file (FIFO queue)
- The read and write operations may become blocked if the queue is empty (on read) and full (on write)
- If a process tries to read before something is written to the pipe, the process is suspended until something is written.
- Pipes exist only as along as the processes which creates them
- When a pipe is closed, the pipe file is destroyed
- The pipe system call finds the first two available positions in the process’s open file table and allocates them for the read and write ends of the pipe.

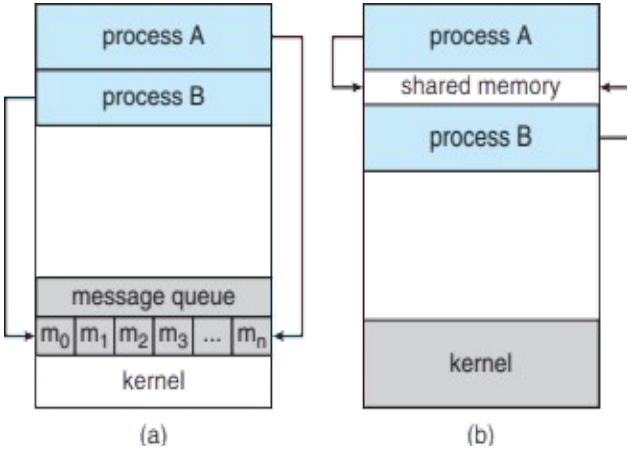
Socket

- A socket is defined as an endpoint for communication
- Concatenation of IP address and port
- The socket 161.25.19.8:1625 refers to port 1625 on host 161.25.19.8
- Communication consists between a pair of sockets

Inter Process Communication

Shared memory

- Processes share memory region for communication
- User processes control communication (not OS)
- For large amount of data transfer
- Issue of process synchronization
- **shmget :** A process use it to create the shared segment in memory
- It returns a segment id
- **shmat :** It is used to access shared memory
- It returns the address of shared memory, enabling a reference
- **Shmdt :** It detaches the shared memory from calling process
- **shmctl :** It sets control parameter or destroys a shared segment



Message passing

- No shared variables
- Establish communication link
- Pass message using send(m) and receive(m)
- For small amount of data transfer (fixed or variable length messages)
- Direct or indirect communication, blocking or non-blocking
- Easier to implement for distributed systems
- Time consuming

PIPE

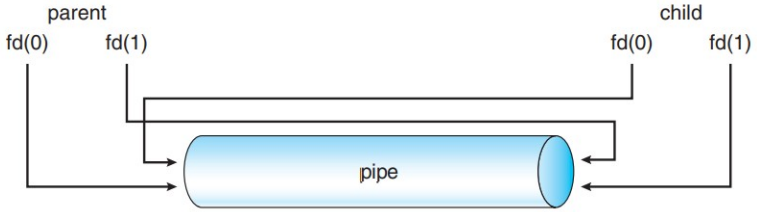
Anonymous Pipe

int pipe (int fd[2])

Create a pipes and places descriptors of write & read

- fd[0] will be the fd(file descriptor) for the read end of pipe.
- fd[1] will be the fd for the write end of pipe.
- Returns : 0 on Success.
- -1 on error.
- Pipe is always unidirectional

UNIX treats a pipe as a special type of file Thus, pipes can be accessed using ordinary read() and write() system calls




```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/wait.h>

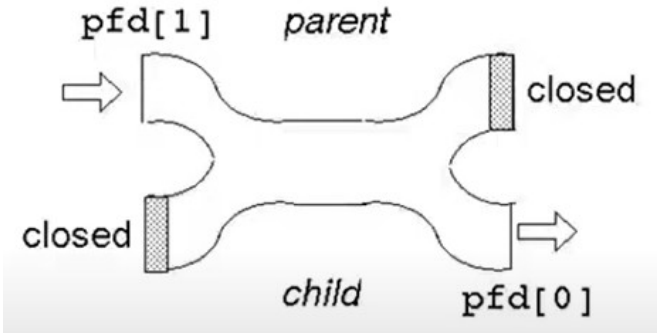
int main(int argc, char const *argv[])
{
        int  fd[2];
        pipe (fd);

        if (fork() == 0)
        {
        char array[10];
        read(fd[0],array,10);
        printf("%s",array);
        }
        else
        {
            write(fd[1],"Helosdfffoo", 6);
            wait(NULL);

        }
}
```

PIPES

Synchronization Mechanism



If PIPE is empty read call will be blocked

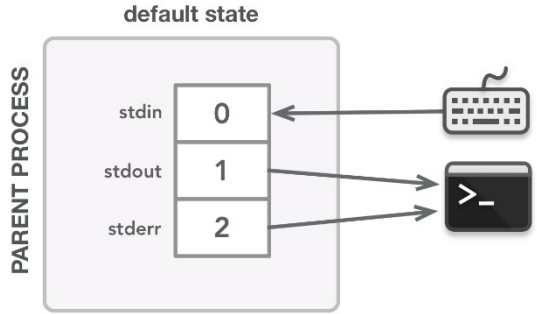
If PIPE is full write call will be blocked

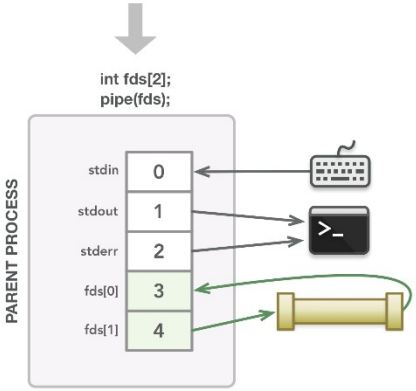
If all write ends are closed for a PIPE read will return 0 EOF

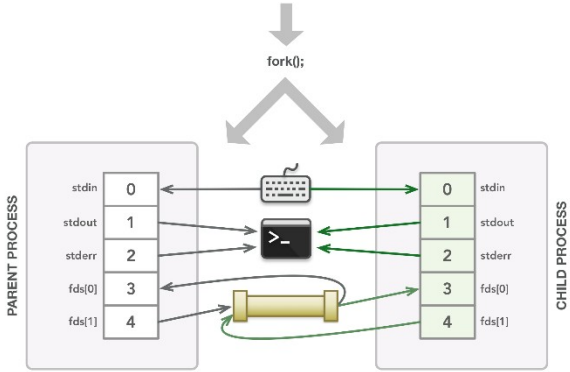
If read ends are closed write will generate SIGPIPE Signal

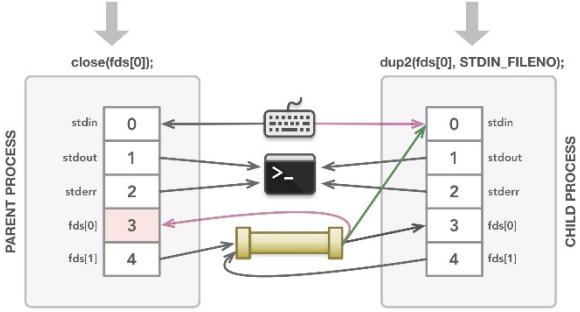
Write and read process's own unused pipe end will prevent it from throwing SIGPIP/returning EOF

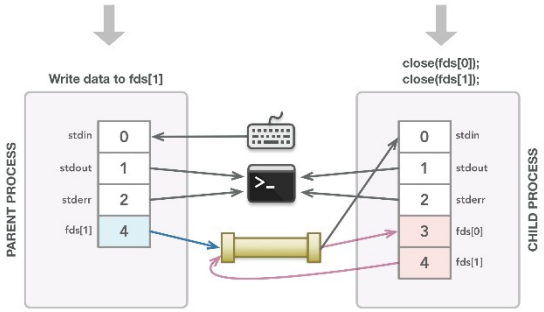
PIPES

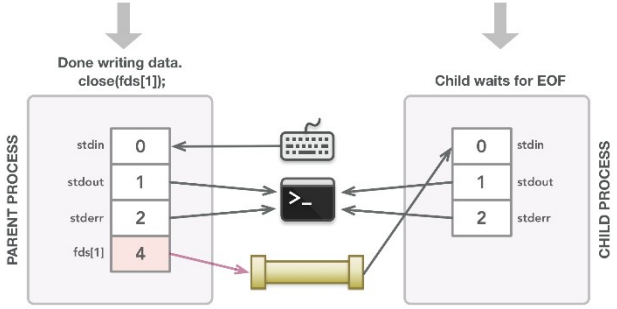

default state
PARENT PROCESS
stdin
stdout
stderr
0
1
2


int fds[2];
pipe(fds);
PARENT PROCESS
stdin
stdout
stderr
fds[0]
fds[1]
0
1
2
3
4

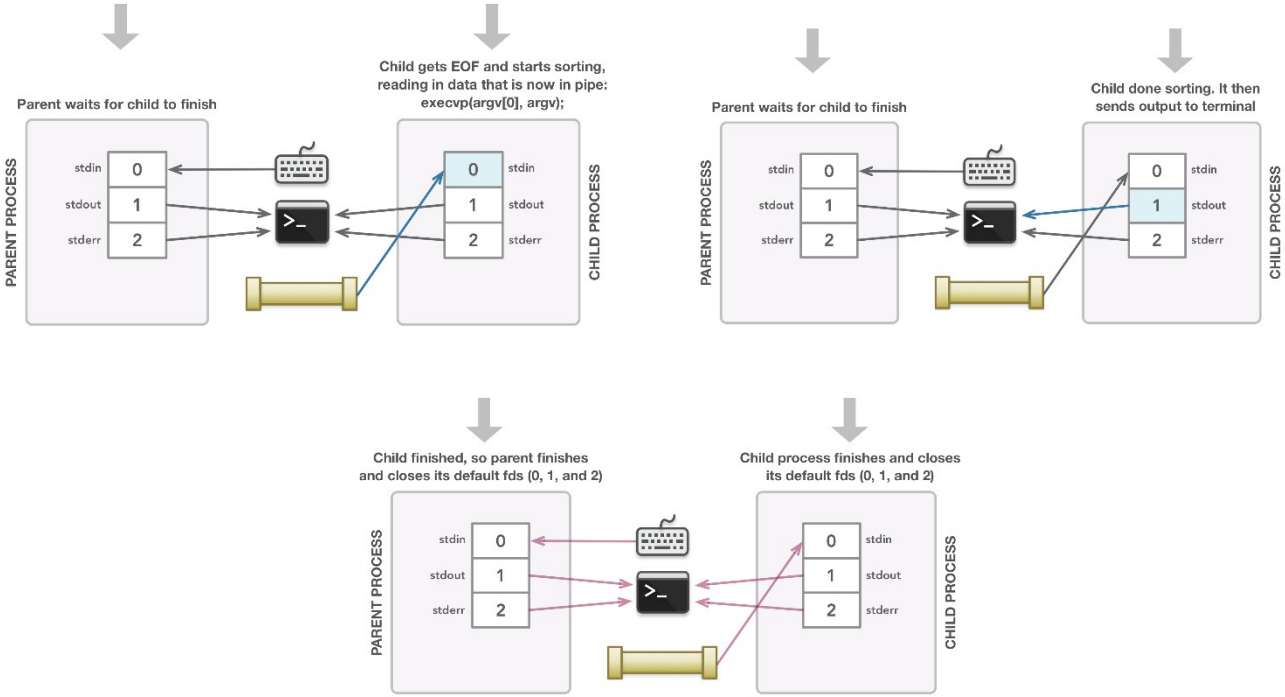

fork();
PARENT PROCESS
CHILD PROCESS
stdin
stdout
stderr
fds[0]
fds[1]
0
1
2
3
4


close(fds[0]);
dup2(fds[0], STDIN_FILENO);
PARENT PROCESS
CHILD PROCESS
stdin
stdout
stderr
fds[0]
fds[1]
0
1
2
3
4


Write data to fds[1]
close(fds[0]);
close(fds[1]);
PARENT PROCESS
CHILD PROCESS
stdin
stdout
stderr
fds[1]
fds[0]
0
1
2
3
4


Done writing data.
close(fds[1]);
Child waits for EOF
PARENT PROCESS
CHILD PROCESS
stdin
stdout
stderr
fds[1]
0
1
2
4

PIPE


Parent waits for child to finish
Child gets EOF and starts sorting, reading in data that is now in pipe: execvp(argv[0], argv);
PARENT PROCESS
CHILD PROCESS
stdin
stdout
stderr
0
1
2
Parent waits for child to finish
Child done sorting. It then sends output to terminal
Child finished, so parent finishes and closes its default fds (0, 1, and 2)
Child process finishes and closes its default fds (0, 1, and 2)

Descriptor table

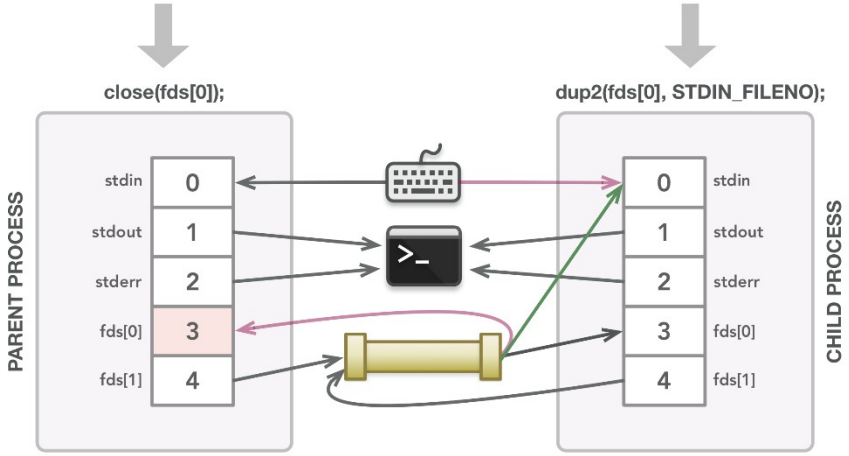
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
  int pipefds[2];

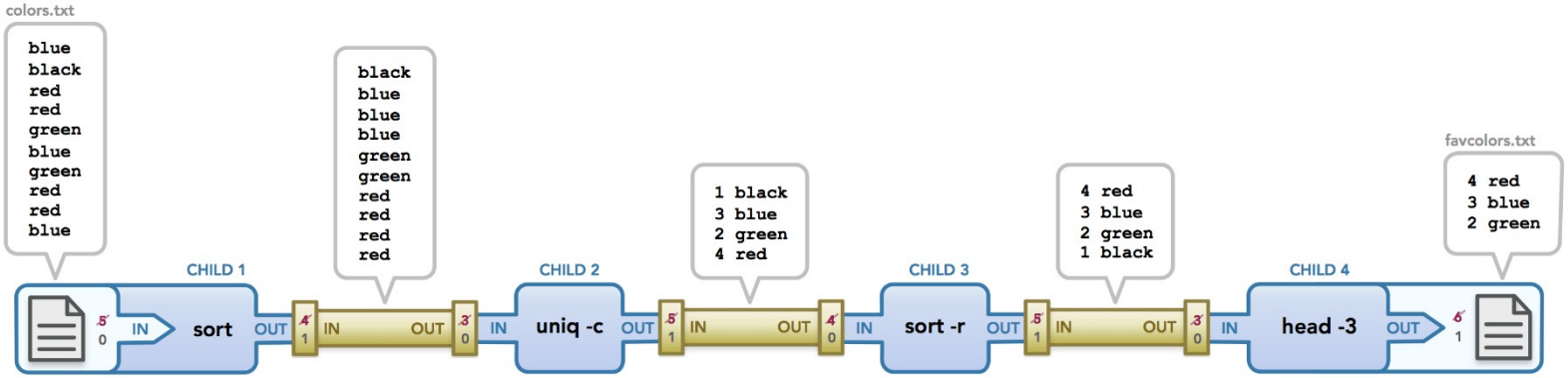
  if(pipe(pipefds) == -1) {
    perror("pipe");
    exit(EXIT_FAILURE);
  }

  printf("Read File Descriptor Value: %d\n", pipefds[0]);
  printf("Write File Descriptor Value: %d\n", pipefds[1]);

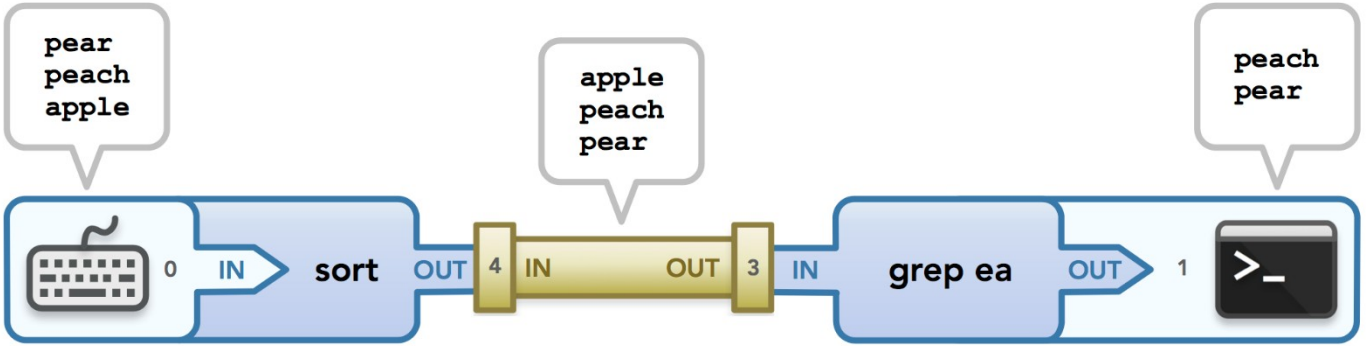
  return EXIT_SUCCESS;
}
```



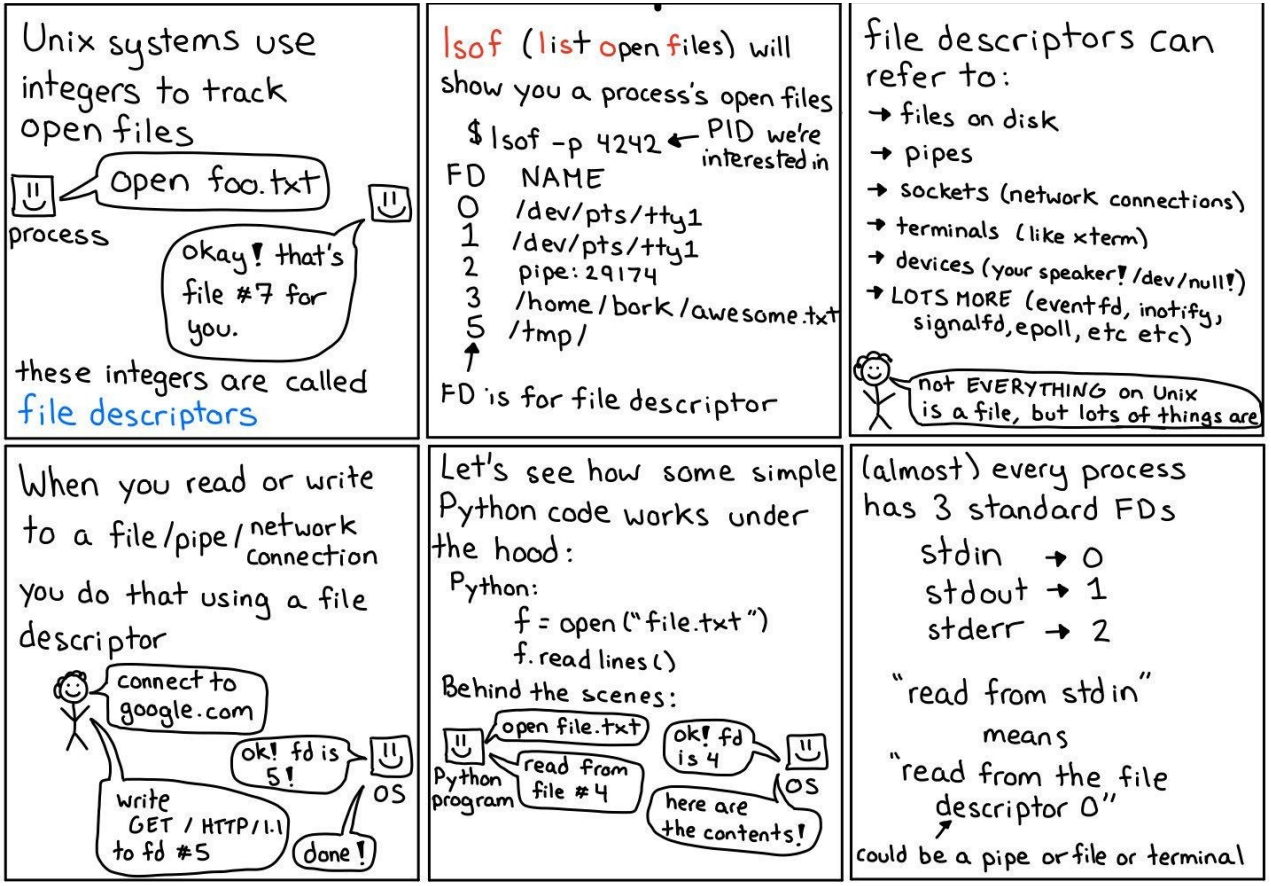
PIPES



sort | grep ea



Descriptor table



Lab

- Run exec and file descriptor



Thank you